

Aufgeschobene Ausführung: Workqueues und Timer

Arbeit entkoppeln, Zeitpunkte planen und Lebensdauer beherrschen

3.03

Aufschub hält kritische Kontexte kurz

AUSLÖSER

Hard-IRQ, Soft-IRQ, Syscall oder anderer Kernelpfad

AUFSCHUB

Work Item oder Timer registrieren

AUSFÜHRUNG

Worker-Task oder Timer-Callback übernimmt

VERTRAG

Daten, Sichtbarkeit und Lebensdauer bleiben gültig

Aufschub ändert den Zeitpunkt und häufig auch den Ausführungskontext.

Mechanismus nach Kontext und Zeitbedarf

WORKQUEUE	Bald ausführen, Callback darf schlafen
DELAYED WORK	Nach grober Verzögerung im Worker ausführen
TIMER_LIST	Timeout in jiffies, kurzer atomarer Callback
HRTIMER	Präziser Zeitpunkt mit ktime_t
TIMER + WORK	Zeitpunkt im Timer, Verarbeitung im Worker

Präzision, Schlafbarkeit und Lebensdauer bestimmen die Auswahl.

Workqueues nutzen verwaltete Worker-Pools

01 Work Item

Beschreibt eine konkrete auszuführende Funktion

02 Workqueue

Definiert Attribute, Flush und Fortschritt

03 Worker Pool

Stellt ausführende Kernel-Threads bereit

04 Worker

Ruft die Work-Funktion im Prozesskontext auf

Eine eigene Workqueue bedeutet nicht automatisch einen eigenen Thread.

work_struct kapselt die spätere Funktion

WORK ITEM

```
static void demo_work(struct work_struct *work)
{
    struct demo *d = container_of(work, struct demo, work);
    mutex_lock(&d->lock);
    process_event(d);
    mutex_unlock(&d->lock);
}

INIT_WORK(&d->work, demo_work);
schedule_work(&d->work);
```

KONTEXT

- Worker-Task
- Schlafen möglich
- current vorhanden

LIFECYCLE

Work Item und Container müssen gültig bleiben.

container_of verbindet den Callback sicher mit seinem Besitzer.

Pending Work koalesziert Queueing-Versuche

QUEUEING

```
if (!queue_work(d->wq, &d->work))
{
    atomic_inc(&d->missed_events);
}

/* Der Worker verarbeitet den aktuellen Zustand,
 * nicht zwingend genau ein Ereignis. */
```

01 Koaleszierung

Pending Work wird nicht erneut eingereicht.

02 Designfrage

Zustand lesen, Ereignisse zählen oder eine eigene Queue führen?

Ein Work Item steht für Arbeit, nicht zwingend für genau ein Ereignis.

Delayed Work verbindet Timer und Worker

DELAYED WORK

```
static DECLARE_DELAYED_WORK(retry_work, retry_fn);

schedule_delayed_work(&retry_work, msecs_to_jiffies(250));

/* erneut terminieren */
mod_delayed_work(system_wq, &retry_work, msecs_to_jiffies(250));
```

GEEIGNET

- Retry
- Debounce
- Periodische Abfrage

NICHT PRÄZISE

Start erst nach Ablauf, der Worker kann zusätzlich warten.

Delayed Work plant eine früheste Startzeit im Worker-Kontext.

Eigene Workqueues definieren Eigenschaften

ALLOKATION

```
d->wq = alloc_workqueue("demo", WQ_UNBOUND | WQ_MEM_RECLAIM, 1);  
if (!d->wq)  
{  
    return -ENOMEM;  
}  
  
queue_work(d->wq, &d->work);
```

STANDARD ZUERST

Die System-Workqueue reicht für viele Treiber aus.

EIGENE QUEUE

Ordering, Reclaim oder Parallelität bewusst steuern.

Eine eigene Queue braucht eine konkrete Eigenschaft als Begründung.

Cancel und Flush lösen andere Aufgaben

`CANCEL_WORK_SYNC`

Abbrechen und laufende Work abwarten

`CANCEL_DELAYED_WORK_SYNC`

Delayed Work korrekt abbrechen

`FLUSH_WORK`

Letzte Queueing-Instanz beenden lassen

`FLUSH_WORKQUEUE`

Alle bei Eintritt vorhandenen Works abwarten

Sync-Varianten schlafen und dürfen keine Callback-Locks halten.

Teardown ordnet Producer, Work und Speicher

01 Producer stoppen

IRQ, Timer oder externe Registrierung deaktivieren

02 Work stoppen

`cancel_work_sync` / `cancel_delayed_work_sync`

03 Queue zerstören

`destroy_workqueue` nach vollständigem Drain

04 Speicher freigeben

Erst wenn kein Callback mehr zugreifen kann

Cancel gehört zum Lifecycle und nicht nur zum Aufräumcode.

timer_list für grobe Timeouts

TIMER CALLBACK

```
static void timeout_fn(struct timer_list *t)
{
    struct demo *d = from_timer(d, t, timer);
    set_bit(TIMED_OUT, &d->flags);
}

timer_setup(&d->timer, timeout_fn, 0);
mod_timer(&d->timer, jiffies + msecs_to_jiffies(500));
```

TIMEOUT

- Günstig
- Jiffies-basiert
- Oft vor Ablauf gelöscht

CALLBACK

- Softirq-Kontext
- Kein Schlafen
- Kurz halten

timer_list passt zu groben Fristen und kurzen atomaren Aktionen.

Periodische Timer und der Sollzeitpunkt

PERIODISCHER HRTIMER

```
static enum hrtimer_restart tick_fn(struct hrtimer *timer)
{
    sample_fast_state();
    hrtimer_forward_now(timer, interval);
    return HRTIMER_RESTART;
}
```

01 Drift vermeiden

Die nächste Frist aus der Zeitbasis ableiten.

02 Überlauf

Callback kürzer als das Intervall halten.

Die Frist folgt der Zeitbasis, nicht der Dauer des letzten Callbacks.

hrtimer liefert präzisere Zeitpunkte

HRTIMER

```
hrtimer_init(&d->timer, CLOCK_MONOTONIC, HRTIMER_MODE_REL);  
  
d->timer.function = tick_fn;  
  
hrtimer_start(&d->timer, ms_to_ktime(10), HRTIMER_MODE_REL);
```

GEEIGNET

- Präziser Zeitpunkt
- ktime_t
- Relative oder absolute Frist

WEITER ATOMAR

Callback kurz halten und aufwendige Arbeit aufschieben.

Höhere Zeitpräzision ändert den atomaren Callback-Kontext nicht.

Timer bestimmt wann, Workqueue bestimmt wo

TIMER UND WORK

```
static void timer_fn(struct timer_list *t)
{
    queue_work(d->wq, &d->work);
}

static void work_fn(struct work_struct *work)
{
    mutex_lock(&d->lock);
    read_and_process(d);
    mutex_unlock(&d->lock);
}
```

TIMER

- Zeitpunkt
- Atomarer Kontext
- Nur anstoßen

WORKQUEUE

- Prozesskontext
- Darf schlafen
- Verarbeitung

Der Timer signalisiert, der Worker verarbeitet.

Timer und Work brauchen Shutdown-Symmetrie

SHUTDOWN

```
/* Producer zuerst dauerhaft stoppen */  
timer_shutdown_sync(&d->timer);  
  
cancel_work_sync(&d->work);  
destroy_workqueue(d->wq);  
kfree(d);
```

REIHENFOLGE

- Timer stilllegen
- Work stoppen
- Queue zerstören
- Speicher freigeben

DEADLOCK VERMEIDEN

Keine Locks halten, die der Callback benötigt.

Erst Producer stoppen, danach Callbacks drainen und Speicher freigeben.

Aufschub braucht einen vollständigen Vertrag

KONTEXT	Darf die spätere Funktion schlafen?
SEMANTIK	Ereignis, Zustand oder koaleszierbares Signal?
ZEITBASIS	Grobe Frist, jiffies oder präzises ktime_t?
PARALLELITÄT	Welche Reihenfolge und max_active gelten?
LIFECYCLE	Wer stoppt Producer und Callbacks?
VERIFIKATION	Cancel, Requeue, Shutdown und Last testen

Zeitpunkt, Kontext und Lifecycle gemeinsam entwerfen.